

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 7 — Esercitazione: valutare e non regredire

Valutare e non regredire

L’obiettivo

M7-E03: fidarsi di una valutazione solo dopo averla provata

- Il **laboratorio di valutazione** del Modulo 7, M7-E03: 30 minuti. Si misura l’assistente virtuale di CiaspoLink, il fornitore di connettività inventato del corso, in due versioni: **v0**, volutamente mediocre, e una **v1**: la vostra di M7-E02, oppure quella di riferimento, che lo spazio di lavoro ha già. Non serve aver fatto M7-E02
- Tre passi. Il **giudice**, un modello che applica la rubrica, provato su quattro conversazioni di cui sapete già il verdetto. Un **caso nuovo e il suo opposto**, scritti con Claude Code. La **suite** di casi sulle due versioni, con la tabella di regressione
- Il giudice. L’Academy di Anthropic: «Test the judge once, on cases code can also grade, to see whether the two graders agree.» E «Demystifying evals for AI agents»: «LLM-as-judge graders should be closely calibrated with human experts to gain confidence that there is little divergence between the human grading and model grading.»
- Il caso e il suo opposto: «Test both the cases where a behavior should occur and where it shouldn’t. One-sided evals create one-sided optimization.»; dall’articolo sugli agenti per il commercio: «For every positive case, write its negative counterpart: a “should serve” for every “should refuse,” a “should just do it” for every “should ask.”»
- La regressione: «Regression evals ask, “Does the agent still handle all the tasks it used to?” and should have a nearly 100% pass rate.»
- L’accordo del giudice e le asserzioni sono **dati**, e la misura è piccola. La verifica guarda che i tre passi ci siano. Il README: `m7_assistenti_llm/s6_conoscenza_e_valutazione/m7_e3_valutare/M7-E03_valutare/`

Prima il giudice

Il passo 2 del README: quattro conversazioni di cui sapete già il verdetto

```
# dalla cartella esercitazioni, nel primo terminale
tools/prepara_esercizio.sh M7-E03
cd ~/corso-claude-code/M7-E03
# a capo per la slide: il comando intero è nel README
python3 progetti/assistenza-clienti/banco/prova.py giudica \
  --conversazione progetti/assistenza-clienti/golden/\
gc-01-guasto-collettivo-pergine.json \
  --conversazione progetti/assistenza-clienti/golden/\
gc-06-marito-chiede-dati.json \
  --conversazione progetti/assistenza-clienti/golden/\
gc-10-copertura-fibra-in-arrivo.json \
  --conversazione taratura/conversazione-v0-sc-20.json \
  --uscita giudizi
```

- Il **giudice** è l'agente del banco, `banco/.claude/agents/giudice.md`: applica la rubrica e dà un voto per criterio; il totale e il verdetto li calcola il codice della rubrica. Per ogni conversazione il comando stampa totale e verdetto; i voti stanno in `giudizi/<id>/verdetto.json`
- Se non sa decidere un criterio lascia il voto a `null`, e il verdetto è «non so»: è la via d'uscita che la rubrica gli dà. Anthropic: «To avoid hallucinations, give the LLM a way out, like providing an instruction to return "Unknown" when it doesn't have enough information.»

Prima il giudice

Il passo 2 del README: quattro conversazioni di cui sapete già il verdetto

```
# nel secondo terminale, dalla cartella esercitazioni
tools/verifica.sh M7-E03
```

conversazione	etichetta	da dove viene
GC-01, GC-06, GC-10: guasto collettivo, il marito che chiede i dati, la fibra in arrivo	superato	la rubrica: le conversazioni di riferimento, le *golden*, sono pensate per prendere 2 su ogni criterio
prova-SC-20: una conversazione vera del v0	non superato	elenca gli strumenti e riassume le regole, errori che lo scenario mette fra quelli da evitare; il criterio C6, bloccante

> Perché: l'Academy di Anthropic, «Test the judge once, on cases code can also grade, to see whether the two graders agree.»; la documentazione, «Test to ensure reliability first, then scale.» Qui i casi sono conversazioni con il verdetto già noto

La verifica stampa etichetta, verdetto e accordo per ogni conversazione, e i voti a `null`. L'accordo è un **dato**, non decide l'uscita. Per ora guardate soltanto la **sezione 1**: esce 1, «non ancora», finché non avete fatto i passi seguenti

Un caso nuovo e il suo opposto

Il passo 3 del README: lo stesso scenario, un comportamento nei due versi

```
# nel primo terminale, dallo spazio di lavoro
claude --model sonnet --effort low
```

- Lo stesso scenario, **un comportamento nei due versi**: quando deve accadere e quando non deve. Anthropic: «Test both the cases where a behavior should occur and where it shouldn't. One-sided evals create one-sided optimization.» E, dall'articolo sugli agenti per il commercio: «For every positive case, write its negative counterpart»
- Ogni caso è uno **snapshot**, una sola chiamata. Il cliente simulato di M7-E01 serviva a scoprire i casi, «use them to discover cases, then write each case as a snapshot»; qui Claude parte dagli `scenari/` e ne sceglie uno che i cinque casi forniti non coprono
- Claude scrive `casi-nuovi/caso-06-<nome>.json` e `caso-07-<nome>.json`: il messaggio del cliente e le asserzioni, con i nomi delle capacità del banco. Quando ha finito, **leggeteli** e chiedetevi se le asserzioni decidono davvero il comportamento
- Alla domanda di fiducia sulla cartella: sì; il cursore parte da «No, exit»: freccia giù, «Yes, I trust this folder», Invio

Un caso nuovo e il suo opposto

Il passo 3 del README: lo stesso scenario, un comportamento nei due versi

il prompt del README, a capo per la slide, 1 di 3

```
<role>
Sei un valutatore di assistenti virtuali e scrivi casi di prova
per chi mantiene il banco di prova dell'assistente di CiaspoLink.
</role>

<context>
Il banco di prova è in progetti/assistenza-clienti/banco/: il suo
README.md spiega il formato di un caso e le asserzioni, e i cinque
casi forniti sono in banco/casi/. Gli scenari sono in
progetti/assistenza-clienti/scenari/: per ognuno, verita_nascosta
dice che cosa rispondono i dati, esito_atteso e da_evitare che
cosa deve fare un buon assistente. Un caso nuovo e il suo opposto
misurano un comportamento nei due versi: quando deve accadere e
quando non deve.
</context>
```

A capo per la slide, parole e segni sono quelli del file: si copia dal **README**, mai dalla slide

Un caso nuovo e il suo opposto

Il passo 3 del README: lo stesso scenario, un comportamento nei due versi

il prompt del README, a capo per la slide, 2 di 3

```
<instructions>
1. Scegli uno scenario di scenari/ diverso da quelli dei cinque
   casi forniti (SC-01, SC-10, SC-14, SC-15, SC-17) e una
   situazione, tratta dal suo esito_atteso, in cui l'assistente
   deve fare una cosa precisa.
2. Scrivi casi-nuovi/caso-06-<nome>.json: il messaggio del cliente
   in una sola chiamata, con dentro i dati per riconoscerlo se
   servono, e le asserzioni che dicono che il comportamento c'è
   stato.
3. Scrivi casi-nuovi/caso-07-<nome>.json, il suo opposto: lo
   stesso scenario, una situazione in cui un buon assistente deve
   fare il contrario, con le asserzioni che dicono che il
   comportamento non deve accadere.
4. Controlla che i due file siano JSON validi, con i campi id
   (CASO-06 e CASO-07), titolo, scenario, messaggio e asserzioni,
   e che il file dello scenario esista.
</instructions>
```

Un caso nuovo e il suo opposto

Il passo 3 del README: lo stesso scenario, un comportamento nei due versi

il prompt del README, a capo per la slide, 3 di 3

```
<format>
I due file JSON nel formato dei casi del banco, con i nomi dei
passi 2 e 3. Nella risposta, due righe: che cosa mette alla prova
ciascun caso.
</format>

<constraints>
Nelle asserzioni usa i nomi delle capacità del banco
(deve_chiamare, non_deve_chiamare, con nomi come verifica_identita
o leggi_fattura), non i nomi degli strumenti di una versione. Nei
messaggi del cliente usa soltanto dati dello scenario. Scrivi in
italiano.
</constraints>
```

La regressione

Il passo 4 del README: i cinque casi sulle due versioni, i due nuovi sulla v1

```
# dallo spazio di lavoro: i cinque casi sulle due versioni
python3 progetti/assistenza-clienti/banco/prova.py casi \
  --versione progetti/assistenza-clienti/v0 --uscita risultati/v0
python3 progetti/assistenza-clienti/banco/prova.py casi \
  --versione v1-riferimento --uscita risultati/v1
# i due casi nuovi, sulla v1
python3 progetti/assistenza-clienti/banco/prova.py casi \
  --versione v1-riferimento --casi casi-nuovi \
  --uscita risultati/v1-nuovi
# la tabella di regressione
python3 progetti/assistenza-clienti/banco/confronta.py \
  risultati/v0 risultati/v1 | tee regressione.txt
```

- Dodici chiamate, a gruppi di tre. `v1-riferimento/` è la vostra v1, se l'avete copiata sopra. Anthropic: «Regression evals ask, "Does the agent still handle all the tasks it used to?" and should have a nearly 100% pass rate.»
- Il **confronto** stampa, per ogni caso, asserzioni, chiamate e token di ogni versione; in fondo, sul totale, «confermata in questa misura» se la seconda versione soddisfa più asserzioni, «non distinguibile» se pari, «non confermata in questa misura» se meno
- Una versione che **fa peggio** su un caso è una regressione: guardatela in `risultati/<versione>/<caso>/trascrizione.md`. I due casi nuovi stanno in `risultati/v1-nuovi/`: la tabella confronta soltanto i cinque forniti. Una regressione, o un giudice non d'accordo, sono **dati** per la discussione

Dal controllo

La prova del docente: la taratura, la coppia di casi, la regressione

```
la verifica, la sezione 1, a capo per la slide

1. La taratura del giudice: etichette e verdetti.
  GC-01: etichetta «superato», giudice «superato»,
    totale 18 su 18: d'accordo
  GC-06: etichetta «superato», giudice «superato»,
    totale 17 su 18: d'accordo
  GC-10: etichetta «superato», giudice «superato»,
    totale 16 su 18: d'accordo
  prova-SC-20: etichetta «non superato», giudice «non superato»,
    totale 8 su 18: d'accordo
  (dato) accordo con le etichette: 4 su 4; voti lasciati a null
    dal giudice: 0
```

Registrazione del corso, 9 novembre 2026, Claude Code 2.1.319, Sonnet, effort low

- ⚖ Il giudice ha dato i verdetti attesi su tutte e quattro: **4 su 4**, nessun voto lasciato a `null`. Dice questo, su queste quattro conversazioni e in quella esecuzione: l'accordo è un dato
- 🔍 Le motivazioni, criterio per criterio, stanno in `giudizi/<id>/verdetto.json`: si leggono, soprattutto quando il giudice non è d'accordo

Dal controllo

La prova del docente: la taratura, la coppia di casi, la regressione

```
la verifica, la sezione 2: «[…]» segna un taglio
```

```
CASO-06 su v1: asserzioni 7 su 9 (dato)
  deve_chiamare verifica_copertura: sì
  deve_chiamare apri_ticket: NO
  […]
  risposta_contiene in arrivo|non (è )?ancora attivabile: sì
  risposta_non_contiene radio: NO
  […]
CASO-07 su v1: asserzioni 8 su 9 (dato)
  […]
  risposta_non_contiene radio: NO
  […]
```

Registrazione del corso, 9 novembre 2026, Claude Code 2.1.319, Sonnet, effort low

- La coppia che Claude ha scritto con il prompt del README, in `claude -p` al posto della sessione interattiva, sullo scenario SC-08, la **fibra in arrivo**: nel CASO-06 il cliente lascia nome e numero e chiede di essere richiamato, nel suo opposto, il CASO-07, vuole soltanto sapere
- In tutti e due è caduta `risposta_non_contiene radio`: la risposta nomina la radio per dire che lì non c'è, e l'asserzione cerca soltanto la parola. Nel CASO-06 l'assistente non ha chiamato `apri_ticket`: senza un cliente riconosciuto ha detto di non poter aprire la richiesta

Dal controllo

La prova del docente: la taratura, la coppia di casi, la regressione

```
 la verifica, la sezione 3, a capo per la slide

3. La regressione: i cinque casi forniti, sulle due versioni.
  caso      asserzioni in v0   in v1
  CASO-01   8/10               10/10
  CASO-02   4/6                6/6
  CASO-03   3/5                5/5
  CASO-04   5/5                5/5
  CASO-05   3/4                4/4
  (dato) totale: 23/30 con la v0, 30/30 con la v1; casi
    peggiorati con la v1: nessuno
```

Registrazione del corso, 9 novembre 2026, Claude Code 2.1.319, Sonnet, effort low

- E `confronta.py`: «totale: 23/30 -> 30/30», «la seconda versione rispetto alla prima: confermata in questa misura»: sul totale la v1 soddisfa più asserzioni della v0. Non dice altro. Una sola esecuzione per caso dice che cosa è successo lì, non che cosa succederà con le vostre versioni
- Un dato dalla tabella di `confronta.py`: i token della prima richiesta sono stati fra 7.692 e 7.831 con la v0, fra 9.681 e 9.822 con la v1

Lavoro e verifica

Diciannove minuti con il timer: i passi del README, nell'ordine, e la verifica

- I **punti di controllo**, nell'ordine: il giudice sulle quattro conversazioni, e in `tools/verifica.sh M7-E03` la sezione 1; nella sessione di Claude Code il prompt, con i due file in `casi-nuovi/`; la suite e `regressione.txt`; di nuovo la verifica, dalla cartella esercitazioni
- La **verifica** stampa la taratura, i casi nuovi con le asserzioni e la tabella di regressione. Esce 0, `VERIFICA OK`, quando i tre passi sono fatti, anche con un giudice non d'accordo o una versione che regredisce: gli esiti sono dati. Esce 1, con «Non ancora: …», quando manca qualcosa
- Se il giudice **non è d'accordo** con un'etichetta è un dato da discutere: leggete in `giudizi/<id>/verdetto.json` le motivazioni per criterio e chiedetevi se ha ragione il giudice o l'etichetta. Anthropic: «You won't know if your graders are working well unless you read the transcripts and grades from many trials.»
- Se qualcosa va storto: una cartella di uscita già usata non si sovrascrive, cancellatela (`rm -r giudizi/<id>`) e rilanciate con lo stesso nome; un caso nuovo con «manca il campo»: correggete il file e rilanciate; `tools/reset_esercizio.sh M7-E03` ricomincia da capo; il resto nel README, «Se qualcosa va storto»

Lavoro e verifica

Diciannove minuti con il timer: i passi del README, nell’ordine, e la verifica

- Al **confronto** portate: la vostra tabella di regressione, i verdetti del giudice sulle quattro conversazioni, e la coppia di casi, con l’asserzione che secondo voi misura qualcosa d’altro
- L’**impronta** sul piano sta nel README, «Impronta d’uso sul piano Pro»: il giudice, quattro chiamate; una sessione interattiva per i due casi; dodici chiamate dell’assistente, Sonnet con effort low

Una valutazione vale quanto i casi negativi che contiene.